Date and TimeDelta in Pandas

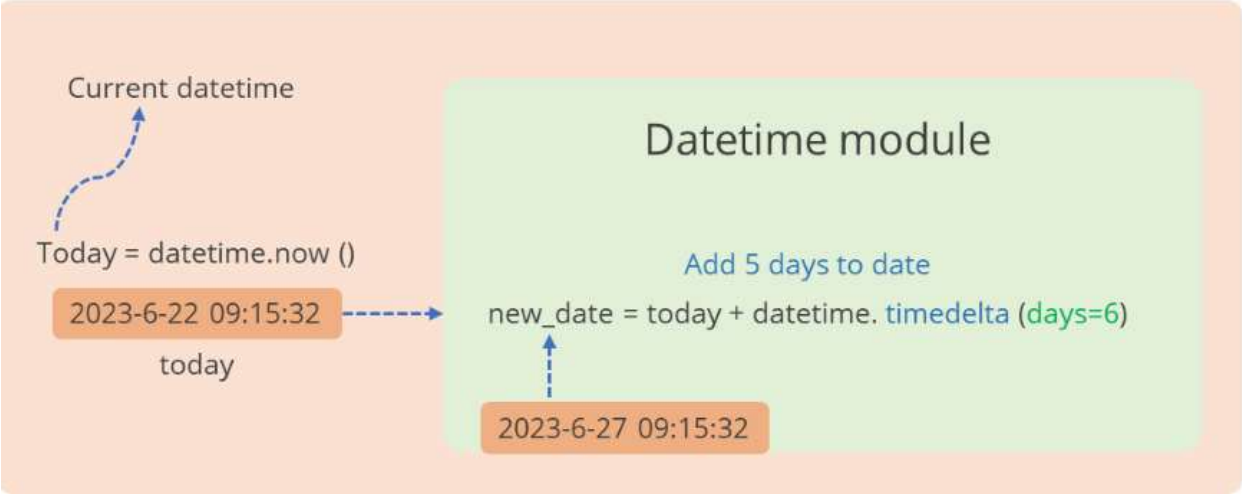
Agenda

In this lesson, we will cover the following concepts with the help of examples:

- Date and TimeDelta in Pandas
- Date Handling in Pandas
  - Extracting Components from Dates
- Timedelta in Pandas
  - Creating a Timedelta
  - Performing Arithmetic Operations
  - Resampling Time Series Data
- Categorical Data Handling
  - Creating a Categorical Variable
  - Counting Occurrences of Each Category
  - Creating Dummy Variables
  - Label Encoding

1. Date and TimeDelta in Pandas

In Pandas, the datetime module provides robust functionality for handling date and time data, while the timedelta class allows for convenient manipulation of time intervals. This combination is particularly useful for time-based analysis and working with temporal data in a DataFrame.



2. Date Handling in Pandas

Creating a Date Range:

- The date_range function is used to generate a sequence of dates within a specified range.
- It is a powerful tool for creating time indices or date columns in a DataFrame.
- The start and end parameters define the range, while freq determines the frequency, such as daily (D) or monthly (M).

```
import pandas as pd

# Generate a date range
date_range = pd.date_range(start='2023-01-01', end='2023-01-10', freq='D')
print(date_range)
```

```
DatetimeIndex(['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04',
               '2023-01-05', '2023-01-06', '2023-01-07', '2023-01-08',
               '2023-01-09', '2023-01-10'],
              dtype='datetime64[ns]', freq='D')
```

2.1 Extracting Components from Dates

Pandas provides the dt accessor to extract various components (Example: day, month, year) from a date column in a DataFrame. This is valuable for time-based analysis when specific date attributes need to be considered.

```
import pandas as pd

# Assuming 'df' is your DataFrame with a 'Date' column
data = {'Date': ['2023-01-01', '2023-02-15', '2023-03-20']}
df = pd.DataFrame(data)
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Extracting day, month, and year information
df['Day'] = df['Date'].dt.day
df['Month'] = df['Date'].dt.month
df['Year'] = df['Date'].dt.year

# Displaying the DataFrame with extracted information
print(df[['Date', 'Day', 'Month', 'Year']])
```

```
        Date  Day  Month  Year
0 2023-01-01    1      1  2023
1 2023-02-15   15      2  2023
2 2023-03-20   20      3  2023
```

```
# Extracting weekday and weekend information
df = pd.DataFrame({'Date': pd.date_range(start='2023-01-01', periods=5)})
df['Weekday'] = df['Date'].dt.weekday
df['IsWeekend'] = df['Date'].dt.weekday // 5 == 1
print(df[['Date', 'Weekday', 'IsWeekend']])
```

```
        Date  Weekday  IsWeekend
0 2023-01-01        6       True
1 2023-01-02        0      False
2 2023-01-03        1      False
3 2023-01-04        2      False
4 2023-01-05        3      False
```

```
# Shifting dates forward or backward
df['Date'] = pd.to_datetime(df['Date'])
df['PreviousDate'] = df['Date'] - pd.Timedelta(days=1)
df['NextDate'] = df['Date'] + pd.Timedelta(days=1)
print(df[['Date', 'PreviousDate', 'NextDate']])
```

```
        Date PreviousDate   NextDate
0 2023-01-01   2022-12-31 2023-01-02
1 2023-01-02   2023-01-01 2023-01-03
2 2023-01-03   2023-01-02 2023-01-04
3 2023-01-04   2023-01-03 2023-01-05
4 2023-01-05   2023-01-04 2023-01-06
```

3. Timedelta in Pandas

3.1 Creating a Timedelta

- The Timedelta class in Pandas represents a duration or the difference between two dates or times.
- It can be created by specifying the desired duration, such as days, hours, or minutes.

```
import pandas as pd

data = {
    'Date': pd.date_range(start='2023-01-01', periods=10, freq='H'),
    'Value1': range(10),
    'Value2': range(10, 20)
}
df = pd.DataFrame(data)

# Creating a timedelta of 3 days
delta = pd.Timedelta(days=3)
```

3.2 Performing Arithmetic Operations

Timedelta objects can be used to perform arithmetic operations on dates. For example, adding a timedelta to a date results in a new date. This is useful for calculating future or past dates based on a given time interval.

```
# Performing arithmetic operations with timedeltas
df['Date'] = pd.to_datetime(df['Date'])
df['FutureDate'] = df['Date'] + pd.Timedelta(weeks=2, days=3, hours=12)
print(df[['Date', 'FutureDate']])
```

```
                 Date          FutureDate
0 2023-01-01 00:00:00 2023-01-18 12:00:00
1 2023-01-01 01:00:00 2023-01-18 13:00:00
2 2023-01-01 02:00:00 2023-01-18 14:00:00
3 2023-01-01 03:00:00 2023-01-18 15:00:00
4 2023-01-01 04:00:00 2023-01-18 16:00:00
5 2023-01-01 05:00:00 2023-01-18 17:00:00
6 2023-01-01 06:00:00 2023-01-18 18:00:00
7 2023-01-01 07:00:00 2023-01-18 19:00:00
8 2023-01-01 08:00:00 2023-01-18 20:00:00
9 2023-01-01 09:00:00 2023-01-18 21:00:00
```

3.3 Resampling Time Series Data

Time series data often comes with irregular time intervals. Resampling is the process of changing the frequency of the time series data, either by upsampling (increasing frequency) or downsampling (decreasing frequency).

```
# Resampling time series data
df.set_index('Date', inplace=True)
df_resampled = df.select_dtypes(include='number').resample('D').sum()
print(df_resampled)
```

```
            Value1  Value2
Date
2023-01-01      45     145
```

4. Categorical Data Handling

4.1 Creating a Categorical Variable

Pandas provides the categorical class to create a categorical variable. Categorical variables are useful when dealing with data that can be divided into distinct, non-numeric categories.

```
import pandas as pd

# Creating a categorical variable
categories = ['Low', 'Medium', 'High']
values = ['Low', 'Medium', 'High', 'Low', 'High']
cat_variable = pd.Categorical(values, categories=categories, ordered=True)
print(cat_variable)
```

```
['Low', 'Medium', 'High', 'Low', 'High']
Categories (3, object): ['Low' < 'Medium' < 'High']
```

4.2 Counting Occurrences of Each Category

The value_counts() method is used to count the occurrences of each category in a categorical column of a DataFrame.

```
# Assuming 'df' is your DataFrame with a 'Category' column
df = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B', 'A']})

# Counting occurrences of each category
category_counts = df['Category'].value_counts()
print(category_counts)
```

```
Category
A    3
B    2
C    1
Name: count, dtype: int64
```

4.3 Creating Dummy Variables

When working with machine learning models or statistical analyses, creating dummy variables is often necessary to represent categorical data numerically. The get_dummies function creates binary columns for each category, effectively converting categorical data into a numerical format.

```
# Assuming 'df' is your DataFrame with a 'Category' column
df = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B', 'A']})

# Creating dummy variables for categorical data
dummy_variables = pd.get_dummies(df['Category'], prefix='Category')
print(dummy_variables)
```

```
   Category_A  Category_B  Category_C
0        True       False       False
1       False        True       False
2        True       False       False
3       False       False        True
4       False        True       False
5        True       False       False
```

4.4 Label Encoding

Another way to handle categorical data is through label encoding, where each category is assigned a unique numerical label. This is useful in scenarios where ordinal relationships exist between categories.

```
# Assuming 'df' is your DataFrame with a 'Category' column
df = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B', 'A']})

# Label Encoding
df['Category_LabelEncoded'] = df['Category'].astype('category').cat.codes
print(df[['Category', 'Category_LabelEncoded']])
```

```
  Category  Category_LabelEncoded
0        A                      0
1        B                      1
2        A                      0
```

```
3        C                    2
4        B                    1
5        A                    0
```

Assisted Practice

Problem Statement:

Analyze the housing dataset with a focus on handling date and categorical data to gain insights into house sales over time and the influence of house characteristics on its price.

Steps to Perform:

- Convert the **YearBuilt** and **YearRemodAdd** columns to datetime format (if not converted)
- Extract useful components from the date like the year, month, or day
- Calculate the time difference between the year the house was built and the year it was remodeled
- Perform necessary arithmetic operations
- Count the number of occurrences of each category in categorical features